Java - Classes and Objects

//Java supports the following fundamental OOPs concepts

- Classes
- Objects
- Inheritance
- Polymorphism
- Encapsulation
- Abstraction
- Instance
- Method
- Message Passing

//What are Java Classes?

A **class** is a blueprint from which individual objects are created (or, we can say a class is a **data type** of an object type). In Java, everything is related to classes and objects. Each class has its **methods** and **attributes** that can be accessed and manipulated through the objects.

For example, if you want to create a class for *students*. In that case, "*Student*" will be a class, and student records (like *student1*, *student2*, etc) will be objects.

We can also consider that class is a factory (user-defined blueprint) to produce objects

//Properties of Java Classes

- A class does not take any byte of memory.
- A class is just like a real-world entity, but it is not a real-world entity. It's a blueprint where we specify the functionalities.
- A class contains mainly two things: Methods and Data Members.
- A class can also be a nested class.
- Classes follow all of the rules of OOPs such as inheritance, encapsulation, abstraction, etc.

Java - Classes and Objects

//Types of Class Variables

A class can contain any of the following variable types.

- **Local variables** − Variables defined inside methods, constructors or blocks are called local variables. The variable will be declared and initialized within the method and the variable will be destroyed when the method has completed.
- **Instance variables** − Instance variables are variables within a class but outside any method. These variables are initialized when the class is instantiated. Instance variables can be accessed from inside any method, constructor or blocks of that particular class.
- **Class variables** − Class variables are variables declared within a class, outside any method, with the static keyword.

// Creating a Java class

```
class Dog {
 // Declaring and initializing the attributes
 String breed;
 int age;
 String color;
 // methods to set breed, age, and color of the dog
 public void setBreed(String breed) {
  this.breed = breed;
 }
 public void setAge(int age) {
  this.age = age;
 }
 public void setColor(String color) {
  this.color = color;
```

```
 }
 // method to print all three values
 public void printDetails() {
  System.out.println("Dog detials:");
  System.out.println(this.breed);
  System.out.println(this.age);
  System.out.println(this.color);
 }
}
```

What are Java Objects?

An **object** is a variable of the type **class**, it is a basic component of an object-oriented programming system. A class has the methods and data members (attributes), these methods and data members are accessed through an **object**. Thus, an object is an instance of a class.

If we consider the real world, we can find many objects around us, cars, dogs, humans, etc. All these objects have a state and a behavior.

If we consider a dog, then its state is - name, breed, and color, and the behavior is - barking, wagging the tail, and running.

If you compare the software object with a real-world object, they have very similar characteristics. Software objects also have a state and behavior. A software object's state is stored in fields, and behavior is shown via methods. So, in software development, methods operate on the internal state of an object, and object-to-object communication is done via methods.

//Creating (Declaring) a Java Object

As mentioned previously, a class provides the blueprints for objects. So basically, an object is created from a class. In Java, the new keyword is used to create new objects.

There are three steps when creating an object from a class –

Java - Classes and Objects

- **Declaration** – A variable declaration with a variable name with an object type.
- **Instantiation** – The 'new' keyword is used to create the object.
- **Initialization** – The 'new' keyword is followed by a call to a constructor. This call initializes the new object.

Note: parameters are optional and can be used while you're using constructors in the class.

```
// Creating a Java class
class Dog {
 // Declaring and initializing the attributes
 String breed;
 int age;
 String color;


 // methods to set breed, age, and color of the dog
 public void setBreed(String breed) {
  this.breed = breed;
 }
 public void setAge(int age) {
  this.age = age;
 }
 public void setColor(String color) {
  this.color = color;
 }
```

Instance variables and methods are accessed via created objects. To access an instance variable, following is the fully qualified path-

Java - Classes and Objects

```
/* First create an object */
ObjectReference = new Constructor();

/* Now call a variable as follows */
ObjectReference.variableName;
```

/* Now you can call a class method as follows */

Create a class named Puppy. In Puppy class constructor, puppy name is printed so that when the object is created, its name is printed. An instance variable puppyAge is added and using getter/setter method, we can manipulate the age. In main method, an object is created using new operator. Age is updated using setAge() method and using getAge(), the age is printed.

```
public class Puppy {
  int puppyAge;
  public Puppy(String name) {
    // This constructor has one parameter, <i>name</i>.
    System.out.println("Name chosen is :" + name );
  }
  public void setAge( int age ) {
    puppyAge = age;
  }
  public int getAge( ) {
    System.out.println("Puppy's age is :" + puppyAge );
    return puppyAge;
  }
  public static void main(String []args) {
    /* Object creation */
```

Java - Classes and Objects

```
    Puppy myPuppy = new Puppy( "tommy" );

    /* Call class method to set puppy's age */

    myPuppy.setAge( 2 );

    /* Call another class method to get puppy's age */

    myPuppy.getAge( );

    /* You can access instance variable as follows as well */

    System.out.println("Variable Value :" + myPuppy.puppyAge );

  }

}
```

Rules for using the Classes and Objects Concepts

Let's now look into the source file declaration rules (to use the Java classes & objects approach). These rules are essential when declaring classes, import statements, and package statements in a source file.

- There can be only one public class per source file.
- A source file can have multiple non-public classes.
- The public class name should be the name of the source file as well which should be appended by **.java** at the end. For example – the class name is *public class Employee{}* then the source file should be as Employee.java.
- If the class is defined inside a package, then the package statement should be the first statement in the source file.
- If import statements are present, then they must be written between the package statement and the class declaration. If there are no package statements, then the import statement should be the first line in the source file.
- Import and package statements will imply to all the classes present in the source file. It is not possible to declare different import and/or package statements to different classes in the source file.